



SECURITY REVIEW

FOR WILD PROTOCOL



PREPARED FOR
Wild Protocol

PREPARED BY
Kann Audits

DATE REPORTED
29 June 2025

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	SCOPE	04
03	AUDITORS	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	H-01 LP Fees Unclaimable via launchV4Pool()	09
	M-01 Bonding Curve Check stepsize * numSteps May Not Match curveSupply	10
	M-02 Incorrect ETH Transfer to State Manager - Full msg.value Sent Instead of Used Amount	11
	M-03 Excess Base Tokens Not Refunded for ERC20 tokens	12
	M-04 ERC20 Transfer - Not All Tokens Return Boolean	13
	L-01 Forced Graduation Flag Ignored	14
	I-01 Missing Validation - numSteps and prices.length Mismatch	15
08	DISCLAIMER	16
09	ABOUT KANN AUDITS	17

01 - OVERVIEW

AUDIT SUMMARY

Wild Protocol engaged Kann Audits to perform a security review of the Wild Protocol smart contracts. A team of 1 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

7

ISSUES

CRITICAL	-
HIGH	01
MEDIUM	04
LOW	01
INFORMATIONAL	01

ENGAGEMENT DETAILS

PREPARED FOR	Wild Protocol
PREPARED BY	Kann Audits
DATE REPORTED	29 June 2025
METHODS	Manual review

02 - WHAT WE REVIEWED

SCOPE

REPOSITORY	https://github.com/wildprotocol/elpee/tree/wip
AUDITED COMMIT	9e66614

#	FILE
01	/src/Deployer.sol
02	/src/LpLocker.sol
03	/src/StateManager.sol
04	/src/libraries/CurrencySend.sol
05	/src/libraries/CurveMath.sol

03 - THE TEAM ON THIS ENGAGEMENT

AUDITORS



Kann

LEAD SECURITY RESEARCHER

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL	● HIGH	● MEDIUM	● LOW	● INFO
-	01	04	01	01
TOTAL 07 FIXED 07 ACKNOWLEDGED 00				

ID	TITLE	SEVERITY	STATUS
H-01	LP Fees Unclaimable via launchV4Pool()	● HIGH	● FIXED
M-01	Bonding Curve Check stepsize * numSteps May Not Match curveSupply	● MEDIUM	● FIXED
M-02	Incorrect ETH Transfer to State Manager - Full msg.value Sent Instead of Used Amount	● MEDIUM	● FIXED
M-03	Excess Base Tokens Not Refunded for ERC20 tokens	● MEDIUM	● FIXED
M-04	ERC20 Transfer - Not All Tokens Return Boolean	● MEDIUM	● FIXED
L-01	Forced Graduation Flag Ignored	● LOW	● FIXED
I-01	Missing Validation - numSteps and prices.length Mismatch	● INFORMATIONAL	● FIXED

DETAILED FINDINGS

H-01

LP Fees Unclaimable via launchV4Pool()

SEVERITY

● HIGH

STATUS

● FIXED

■ DESCRIPTION

The function `launchV4Pool()` allows custom pool creation and token graduation (with manually set ticks and spacing). However, it does not call `lpLocker.setTokenParams()`, which is required to configure the LP locker for fee claiming.

Without calling `setTokenParams()`, fees earned from the pool become unclaimable, rendering the fee distribution logic non-functional for tokens launched via `launchV4Pool()`.

In contrast, the `graduateToken()` function does invoke `lpLocker.setTokenParams()`, meaning fee claiming only works when using that path. This makes the `launchV4Pool()` function incomplete and effectively useless for real-world deployments where fees matter.

■ REMEDIATION

● FIXED

M-01

Bonding Curve Check stepsize * numSteps May Not Match curveSupply

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

When launching a token with bonding curve parameters, there is no validation ensuring that $\text{stepsize} * \text{numSteps} == \text{curveSupply}$. If this is not aligned:

If $\text{stepsize} * \text{numSteps} > \text{curveSupply}$, the bonding curve will appear to offer more tokens than were actually allocated, leading to potential inconsistencies (e.g., using LP pool tokens to fulfill purchases).

If $\text{stepsize} * \text{numSteps} < \text{curveSupply}$, part of the curve supply will become unreachable via the bonding curve, and remain unused.

While this behavior may be intentional to allow flexible bonding curve shapes, without an explicit check or warning, it can lead to unintended launch behavior due to misconfiguration.

■ REMEDIATION

● FIXED

M-02

Incorrect ETH Transfer to State Manager - Full msg.value Sent Instead of Used Amount

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

When a user purchases a launch token using ETH (base token), the `acceptAmount()` function correctly calculates the amount used (`amountInUsed`) and refunds the excess ETH back to the user. However, despite the refund, the entire `msg.value` is still forwarded to the `StateManager.buyToken()` call, rather than just the used portion.

This results in the `StateManager` contract receiving more ETH than it should, creating incorrect accounting and potential fund mismanagement.

■ REMEDIATION

● FIXED

M-03

Excess Base Tokens Not Refunded for ERC20 tokens

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

When a user purchases tokens using an ERC20 base token, the function `_getBuyQuoteAndFees()` correctly calculates the actual amount used (`amountInUsed`) and the associated fees. However, if the user sends more than `amountInUsed`, the excess tokens are not refunded.

In contrast, when using ETH as the base token, any overpayment is explicitly refunded. This inconsistent behavior creates a silent loss of funds for users interacting via ERC20 tokens.

■ REMEDIATION

● FIXED

M-04

ERC20 Transfer - Not All Tokens Return Boolean

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

Uses ERC20(token).transfer(...) and checks whether it returns true to confirm success. However, some widely-used tokens like USDT do not return any value on transfer, which causes the boolean check to fail and revert even if the transfer was successful.

■ REMEDIATION

● FIXED

L-01

Forced Graduation Flag Ignored

SEVERITY

● LOW

STATUS

● FIXED

■ DESCRIPTION

Although the deployer can set `allowForcedGraduation` to false, `graduateToken()` and `_launchV4Pool()` do not validate this setting, allowing owner of deployer and creator to forcibly graduate a token even when it was explicitly disabled.

■ REMEDIATION

● FIXED

I-01

Missing Validation - numSteps and prices.length Mismatch

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

■ DESCRIPTION

In the PriceCurve bondingCurveParams, no check ensures that numSteps equals prices.length. A mismatch could result in unexpected pricing behavior or runtime errors.

■ REMEDIATION

● FIXED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits